

Tối ưu và nén mô hình (Model Optimization and Compression)

PHỤ THUỘC B/11-gioi-han-tinh-toan (bắt buộc — *roofline*, memory-bound so với compute-bound; không có nó thì chương này chỉ là danh sách mẹo); B/09-thiet-ke-ham-mat-mat (phân phối mềm dùng trong chứng cất); C/18/02-kinh-te-hoc-pretraining (cùng logic ngân sách, đổi cột chi); D/19-numpy-toi-pytorch (AMP, channels_last, profiler).

TÓM TẮT

Mô hình tốt nhất của bạn không vừa ràng buộc triển khai. Chương này trình bày năm đòn bẩy để ép nó vừa — giảm độ phân giải, đổi kiến trúc, lượng tử hoá, chưng cất, tỉa — và lập luận rằng *thứ tự* dùng chúng quan trọng hơn kỹ thuật của từng cái. Đọc xong bạn lập được đường Pareto độ chính xác–độ trễ cho bài toán của mình và chọn một điểm có lý do, thay vì thử ngẫu nhiên rồi giữ cái nào trông ổn. Nếu chỉ nhớ một điều: FLOPs là đại diện tồi cho độ trễ, nên mỗi kỹ thuật nén phải kèm phép đo trên *đúng phân cứng đích*, và sai lệch giữa FLOPs và thời gian thật đi theo hướng khó đoán.

KIẾN THỨC CẦN CÓ TRƯỚC

- **độ trễ (latency) và thông lượng (throughput)** — độ trễ là thời gian một yêu cầu đi từ vào tới ra; thông lượng là số yêu cầu xử lý được mỗi giây. Gộp batch làm thông lượng tăng và độ trễ tăng — hai đại lượng này thường xung đột.
- **FLOPs** — số phép tính dấu phẩy động của một **lượt xuôi** (*forward pass*). Nó dễ tính trên giấy nhưng chỉ tương quan lỏng lẻo với thời gian thật, vì phần lớn mạng bị chặn bởi băng thông bộ nhớ chứ không bởi năng lực tính toán.
- **lượng tử hoá (quantization)** — biểu diễn trọng số và kích hoạt bằng số nguyên ít bit (thường INT8) thay vì dấu phẩy động 32 bit, để giảm số byte phải nạp và tăng thông lượng; cái giá là sai số làm tròn tích lũy qua các tầng.
- **calibration set** — một tập ảnh nhỏ (thường vài trăm) chạy qua mô hình để đo dải giá trị thực tế của kích hoạt, từ đó chọn hệ số tỉ lệ cho lượng tử hoá. Nó phải *đại diện* cho dữ liệu triển khai, nếu không dải đo được sẽ sai.
- **tỉa (pruning) và sparsity** — đặt một phần trọng số về không. *Sparsity* là tỉ lệ trọng số bằng không. Tỉa *không cấu trúc* rải các số không khắp nơi; tỉa *có cấu trúc* bỏ nguyên kênh hoặc nguyên khối.
- **chưng cất tri thức (knowledge distillation)** — huấn luyện một mô hình nhỏ (student) bắt chước đầu ra của một mô hình lớn (teacher), thay vì chỉ học từ nhãn cứng. Phân phối xác suất mềm của teacher mang nhiều thông tin hơn một nhãn đúng/sai.
- **độ phân giải đầu vào** — kích thước ảnh đưa vào mạng, ví dụ 640×640 . Chi phí tính toán của phần lớn mạng thị giác tỉ lệ với số điểm ảnh, tức tỉ lệ với *bình phương* cạnh.
- **phần cứng đích** — đúng con chip và đúng cấu hình mà mô hình sẽ chạy khi triển khai — Jetson Orin ở chế độ 30 W không phải cùng một cỗ máy với chính nó ở chế độ 15 W, và cả hai đều không phải là chiếc RTX trên bàn bạn.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn tĩa bỏ 90% trọng số, mô hình giữ nguyên độ chính xác, và nó không chạy nhanh hơn một mili-giây nào — phần cứng đã từ chối món quà đó ở chỗ nào?
- Q2. Vì sao kỹ thuật thô nhất trong cả chương — hạ độ phân giải đầu vào — lại được xếp đứng trước mọi kỹ thuật nén tinh vi?
- Q3. Một tầng có đúng *một* kênh trọng số bất thường; vì sao chừng đó đủ để đẩy phần còn lại của tầng xuống dưới ba bit hiệu dụng sau khi lượng tử hoá?
- Q4. Mô hình A ít FLOPs hơn mô hình B nhưng chạy chậm hơn trên cùng một Jetson — FLOPs đã nói dối theo bao nhiêu cách khác nhau?

1.1 Đặt vấn đề

Câu hỏi dẫn dắt của chương rất cụ thể: *mô hình tốt nhất của bạn không vừa ràng buộc triển khai — làm gì tiếp?* Bạn có năm đòn bẫy, chúng không tương đương nhau, và thứ tự dùng chúng quan trọng hơn kỹ thuật của từng cái.

Điều làm chương này khác một danh sách mẹo là chỗ nó xuất phát từ một sự thật vật lý đã có ở B/11-gioi-han-tinh-toan: phần lớn mạng thị giác không bị chặn bởi năng lực tính toán mà bởi *băng thông bộ nhớ*. Từ sự thật đó suy ra gần như mọi thứ còn lại. Lượng tử hoá có tác dụng vì nó giảm số byte phải nạp chứ không phải vì nó giảm số phép nhân. Tĩa không cấu trúc thất bại vì nó giảm số phép nhân mà không giảm số byte phải nạp. Hợp nhất kernel có tác dụng vì nó cắt bớt các lượt đi về bộ nhớ. Ai chưa nắm trực này sẽ học chương này như học một danh sách; ai nắm rồi thì tự suy ra được phần lớn nội dung.

→ **ràng buộc bởi** B/11 — phần lớn mạng thị giác bị chặn bởi băng thông bộ nhớ chứ không bởi năng lực tính, và chính sự thật đó quyết định kỹ thuật nén nào có tác dụng.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn cầm một ngân sách mili-giây và một mô hình quá chậm. Trong một buổi bạn vạch được đường Pareto độ chính xác–độ trễ, chọn một điểm, và giải thích được vì sao chọn nó. Cách cũ là thử ba kỹ thuật ngẫu nhiên rồi giữ lại cái nào trông ổn.

TRỰC GIÁC

Lượng tử hoá *theo tensor* giống việc chọn một khẩu độ và một tốc độ màn trập duy nhất cho cả khung hình có một bóng đèn sáng chói trong đó. Bóng đèn ép bạn phải khép sáng lại, và mọi vùng còn lại chìm xuống sát sàn nhiều — không phải vì máy ảnh kém, mà vì bạn chỉ có một dải động và đã tiêu nó cho một điểm.

Lượng tử hoá *theo kênh* là chụp mỗi vùng bằng một thiết lập riêng rồi ghép lại. Cùng số bit, nhưng dải động được phân bổ theo nhu cầu thật của từng vùng. Đây không phải ẩn dụ lỏng lẻo. Cả hai trường hợp đều là bài toán ánh xạ một dải giá trị liên tục vào một số mức rời rạc cố định. Và ở cả hai, *một giá trị ngoại lai duy nhất cũng đủ phá hỏng độ phân giải của tất cả những giá trị còn lại*.

1.2 Thứ tự thử nghiệm, và lý do của thứ tự đó

A. Bảng thứ tự

Bảng 1.1 là kết luận thực dụng của cả chương, đặt lên trước để bạn dùng được ngay; phần còn lại giải thích từng dòng.

Bảng 1.1. Năm đòn bẩy nén, theo thứ tự nên thử. Cột “vì sao trước” mới là nội dung; cột “cái giá” là thứ bạn phải chấp nhận.

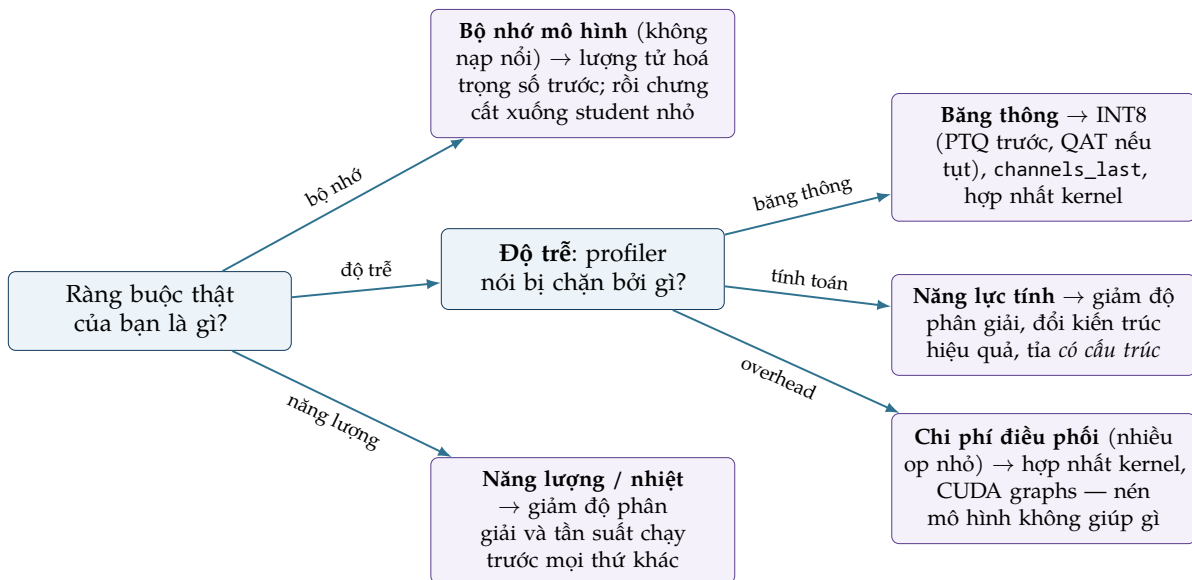
#	Đòn bẩy	Vì sao đứng trước	Cái giá
1	Giảm độ phân giải hoặc độ dài đầu vào	Rẻ nhất, hiệu quả gần như bậc hai theo cạnh ảnh, đảo ngược được ngay trong một dòng config	Mất vật nhỏ và chi tiết mảnh — đúng thứ bạn có thể không đo được trên tập test hiện có
2	Đổi sang kiến trúc hiệu quả	Tối ưu từ thiết kế luôn tốt hơn nén sau; cùng độ chính xác nhưng ít byte hơn ngay từ đầu	Phải huấn luyện lại từ đầu
3	Lượng tử hoá	Tỉ lệ lợi ích trên công sức cao nhất trong nhóm nén; gỡ đúng nút thắt bằng thông	Mất độ chính xác nhỏ, và phụ thuộc mạnh vào calibration đúng
4	Chưng cất	Giữ được nhiều chất lượng nhất trên mỗi đơn vị kích thước	Cần teacher và một chu kỳ huấn luyện đầy đủ
5	Tỉa	Thường <i>không</i> cho tốc độ thật trên phần cứng phổ thông	Công sức lớn, lợi ích không chắc

B. Vì sao thứ tự này chứ không phải thứ tự khác

Ba nguyên tắc sắp xếp, và cả ba đều là nguyên tắc kỹ thuật chứ không phải sở thích:

- **Đảo ngược được thì thử trước.** Giảm độ phân giải là một dòng config; nếu hỏng, bạn biết trong nửa giờ. Chưng cất là một tuần; nếu hỏng, bạn mất một tuần. Xếp theo chi phí của việc *sai* chứ không theo chi phí của việc *đúng*.
- **Thiết kế thắng nén.** Một mạng được thiết kế để có ít byte đi qua luôn tốt hơn một mạng to bị ép nhỏ lại, vì nén sau chỉ bỏ bớt thứ đã có chứ không đổi được cấu trúc luồng dữ liệu.
- **Ưu tiên đòn bẩy gỡ đúng nút thắt.** Nếu bạn memory-bound thì lượng tử hoá gỡ đúng chỗ; tỉa không cấu trúc thì không, dù nó nghe hứa hẹn hơn.

Hình 1.1 chuyển ba nguyên tắc đó thành một cây quyết định theo *ràng buộc* bạn đang bị chặn, vì trong thực tế bạn hiếm khi bị chặn bởi cả ba thứ cùng lúc.



Hình 1.1. Chọn đòn bẩy theo ràng buộc đang chặn bạn, không theo danh sách. Nhánh dưới cùng bên phải là nhánh hay bị bỏ sót nhất: khi thời gian bị nuốt bởi chi phí điều phối kernel, mọi kỹ thuật *nén* đều vô ích và việc phải làm là hợp nhất.

Ví dụ nhỏ nhất: đòn bẩy số 1 rẻ tới mức nào

Chi phí tính của phần lớn mạng tích chập tỉ lệ với số điểm ảnh. Hạ độ phân giải từ 640×640 xuống 512×512 cho tỉ lệ $(512/640)^2 = 0,64$. Tức là cắt 36% chi phí bằng một dòng config, không huấn luyện lại, và hoàn tác được tức thì. Không kỹ thuật nào khác trong chương có tỉ lệ lợi ích trên công sức gần bằng. Cái giá cũng cụ thể như vậy: một vật cao 20 điểm ảnh ở 640 chỉ còn 16 điểm ảnh ở 512, và nếu tập test của bạn không có vật nhỏ thì bạn sẽ *không đo được* thiệt hại — một trường hợp điển hình của việc đánh giá che giấu chế độ hỏng, đúng chủ đề của B/07-chia-du-lieu-va-danh-gia.

↪ **ràng buộc bởi** B/07 — thiệt hại của việc giảm độ phân giải tập trung ở nhóm vật nhỏ, nên nó vô hình nếu tập đánh giá không được phân tách theo nhóm.

1.3 MẠCH VẤN ĐỀ → GIẢI PHÁP

Sáu nút thắt của việc đưa một mô hình vào ngân sách

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

fp32 lãng phí bằng thông.

lượng tử hoá: giảm số bit thì giảm luôn lượng byte phải nạp, và trên phần cứng có đơn vị số nguyên chuyên dụng thì giảm cả thời gian tính.

phân bố giá trị đủ “gọn” để một hệ số tỉ lệ mô tả được.

sai số làm tròn tích lũy, và độ nhảy rất cao với việc chọn hệ số tỉ lệ.

2 Vấn đề

Giải pháp

Vấn đề mới

nhiều trọng số vô dụng.

tĩa.

phần cứng phổ thông không tận dụng được các số không rải rác, nên tiết kiệm trên giấy không thành tốc độ thật. Chỉ tia có cấu trúc, hoặc mẫu 2:4 trên phần cứng hỗ trợ, mới cho lợi ích đo được.

3 Vấn đề

mô hình nhỏ huấn luyện từ đầu thì yếu.

- Giải pháp** chưng cất: dùng phân phối mềm của teacher làm mục tiêu, giàu thông tin hơn nhãn cứng nhiều lần [1].
- Giá phải trả** cần teacher và một chu kỳ huấn luyện đầy đủ.
- 4 Vấn đề** nén sau luôn thua thiết kế tốt.
- Giải pháp** kiến trúc hiệu quả — tích chập tách theo chiều sâu, khối *inverted residual*, và tái tham số hoá kiểu RepVGG.
- Hệ quả** đây lại là mẫu hình “đẩy chi phí sang training-time” đã gặp ở chương 17 và sẽ gặp lại ở chương 21.
- 5 Vấn đề** chọn kiến trúc bằng tay là thủ công và thiên lệch.
- Giải pháp** NAS — tự động tìm kiến trúc.
- Giá phải trả** chi phí tìm kiếm thường lớn hơn lợi ích, và kết quả gắn chặt với phần cứng dùng để tìm, nên hết giá trị khi bạn đổi chip.
- 6 Vấn đề** cùng FLOPs nhưng khác tốc độ.
- Giải pháp** hợp nhất kernel để cắt số lượt đi về bộ nhớ, và bố cục bộ nhớ đúng (`channels_last`, căn lề, tận dụng cache).
- Hệ quả** đây là B/11-gioi-han-tinh-toan áp dụng trực tiếp, và là lý do hai mô hình cùng FLOPs có thể chênh nhau hai lần về thời gian thật.

A. Lượng tử hoá — chi tiết quyết định là hệ số tỉ lệ

Lượng tử hoá đối xứng INT8 ánh xạ một số thực w thành số nguyên $q = \text{round}(w/s)$ với $s = \max |w|/127$, và khôi phục bằng $\hat{w} = s q$. Công thức này nói điều gì? Rằng bạn có đúng 255 mức để phủ toàn bộ dải giá trị, và $\max |w|$ — giá trị lớn nhất, tức giá trị ngoại lai nếu có — quyết định độ mịn của tất cả các mức còn lại.

Ví dụ nhỏ nhất tính tay được. Một tầng có trọng số nằm trong $[-0,1; 0,1]$ ở mọi kênh, trừ một kênh duy nhất có trọng số lên tới 2,0.

- **Per-tensor:** $s = 2,0/127 = 0,01575$. Một trọng số 0,1 được ánh xạ thành $\text{round}(0,1/0,01575) = 6$. Vậy toàn bộ dải $[0; 0,1]$ chỉ còn *sáu* mức — tức chưa tới 3 bit hiệu dụng cho phần lớn mạng.
- **Per-channel:** mỗi kênh có s riêng, nên kênh bình thường dùng $s = 0,1/127$ và trọng số 0,1 được ánh xạ thành 127 — đủ 8 bit.

Cùng số bit, khác nhau hơn một bậc độ lớn về độ phân giải hiệu dụng. Đó là lý do per-channel gần như luôn là lựa chọn đúng cho trọng số của conv và linear, và là lý do một tầng duy nhất có phân bố lệch cũng đủ làm hỏng cả mô hình sau khi lượng tử hoá [2], [3].

Bảng 1.2. Ba chế độ lượng tử hoá thường gặp.

Chế độ	Cơ chế	Chi phí	Hỏng khi nào
Weight-only	Chỉ nén trọng số, kích hoạt vẫn fp16	Gần bằng không	Không giúp gì nếu nút thắt là kích hoạt chứ không phải trọng số
PTQ (post-training)	Chạy calibration set để đo dải kích hoạt, rồi chốt hệ số tỉ lệ	Vài phút	Calibration set không đại diện; mô hình có kích hoạt ngoại lai mạnh
QAT (quantization-aware)	Mô phỏng làm tròn ngay trong lúc huấn luyện để mạng học cách chịu đựng	Một chu kỳ huấn luyện	Không có dữ liệu huấn luyện; hoặc bạn dùng nó khi PTQ vốn đã đủ tốt

Kinh nghiệm thực hành đáng tin: thử PTQ trước, đo, và chỉ chuyển sang QAT khi PTQ tụt quá ngưỡng chấp nhận. Và nguyên nhân số một của việc tụt bất thường không phải thuật toán mà là *calibration set chọn sai*. Vài trăm ảnh lấy toàn từ một điều kiện chiếu sáng cho dài kích hoạt hẹp hơn thực tế. Mọi giá trị nằm ngoài dải đó bị cắt cụt khi triển khai.

B. Tỉa — vì sao 90% sparsity không nhanh hơn

Đây là chỗ trực giác đánh lừa mạnh nhất, nên đáng nói thẳng là bức tranh phức tạp chứ không đơn giản hoá được. Tỉa không cấu trúc đạt được sparsity rất cao mà gần như không mất độ chính xác [4], [5], nhưng *không* nhanh hơn trên GPU thông thường, vì ba lý do cộng lại: các số không rải rác không cho phép bỏ qua việc nạp dữ liệu (bạn vẫn phải đọc chỉ số để biết chỗ nào là không); kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa; và lưu chỉ số của các phần tử khác không tự nó đã tốn bằng thông.

Bảng 1.3. Ba mức cấu trúc của tỉa. Chỉ hai dòng dưới cho tốc độ thật.

Kiểu	Cơ chế	Tốc độ thật	Hỏng / hạn chế
Không cấu trúc	Đặt từng trọng số nhỏ về không	Gần như không đổi trên GPU phổ thông	Chỉ có ích khi bạn cần giảm <i>dung lượng lưu trữ</i> , không phải thời gian
2:4 (bán cấu trúc)	Trong mỗi nhóm 4 trọng số, đúng 2 bằng không	Có, nhưng chỉ trên phần cứng có đơn vị sparse tensor	Ràng buộc cứng làm mất nhiều độ chính xác hơn tỉa tự do cùng tỉ lệ
Có cấu trúc	Bỏ nguyên kênh, nguyên đầu attention, nguyên khối	Có, trên mọi phần cứng — vì mạng thật sự nhỏ đi	Mất nhiều độ chính xác hơn; cần fine-tune lại sau khi tỉa

C. Chưng cất — vì sao phân phối mềm đáng giá

Nhãn cứng nói “đây là chó”. Phân phối mềm của teacher nói “0,7 chó, 0,2 sói, 0,08 cáo, 0,02 mèo” — và ba con số cuối chính là thông tin về *cấu trúc lớp* mà nhãn cứng vứt đi. Student học được rằng sói gần chó hơn mèo, một quan hệ mà nó sẽ phải tự phát hiện lại nếu chỉ có nhãn cứng. Đó là lý do một student nhỏ được chưng cất thường thắng chính student đó huấn luyện từ nhãn cứng, dù cả hai có cùng số tham số.

↔ **cùng nguyên lý với B/09** — phân phối mềm mang thông tin về cấu trúc lớp mà nhãn cứng vứt đi — cùng lý do khiến làm mượt nhãn có tác dụng.

Ba biến thể, theo mức mạnh tăng dần cho thị giác: chưng cất **logit** (đơn giản nhất), chưng cất đặc trưng trung gian (thường mạnh hơn cho thị giác vì nó ép cả biểu diễn chứ không chỉ đầu ra), và chưng cất **attention** cho **transformer** [6]. Xu hướng tại thời điểm viết là chưng cất một mô hình nền tảng lớn xuống student cỡ 20–40M cho thiết bị biên; đây là cách phổ biến nhất để đưa các mô hình lớp SAM hoặc DINO lên thiết bị — ghi nhận này là quan sát thực hành, không phải một kết quả đo mà tôi kiểm chứng được.

D. Kiến trúc hiệu quả, tái tham số hoá, và NAS

Nén sau luôn thua thiết kế tốt, vì thiết kế đổi được cấu trúc luồng dữ liệu còn nén thì không. Dòng MobileNet/ShuffleNet/EfficientNet [7], [8] là các cách khác nhau để giảm số byte đi qua trên mỗi đơn vị sức chứa.

Đáng chú ý riêng là **tái tham số hoá** (*re-parameterization*) kiểu RepVGG: huấn luyện với một khối nhiều nhánh (dễ tối ưu), rồi *gộp toán học* các nhánh thành một tích chập đơn khi triển khai. Vì phép gộp là chính xác chứ không xấp xỉ, bạn được cả hai: động lực học huấn luyện tốt và đồ thị suy luận

gọn. Đây đúng là mẫu hình “đẩy độ khó sang lúc huấn luyện” của B/03-giai-phau-he-thong, ở dạng thuần khiết nhất.

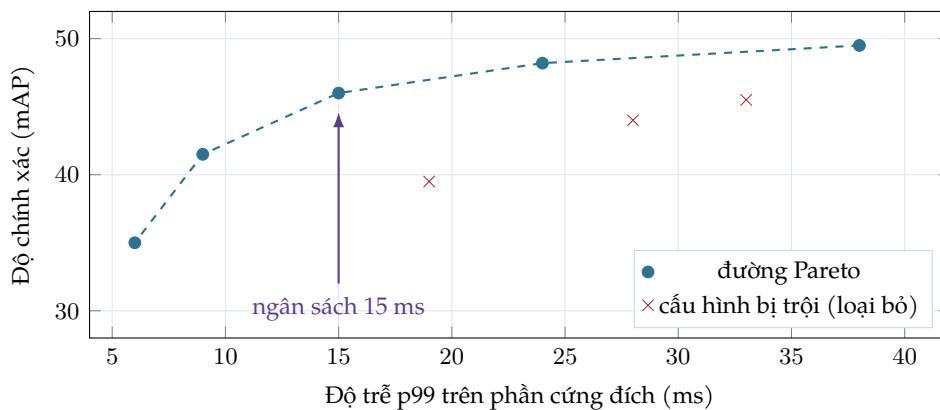
→ cùng nguyên lý với B/03 — độ khó không biến mất mà bị đẩy đi chỗ khác — ở đây là đẩy từ lúc suy luận sang lúc huấn luyện, dạng thuần khiết nhất của nguyên lý đó.

Về NAS thì phát biểu trung thực nhất là: nó hoạt động, nhưng phần lớn dự án không nên dùng. Chi phí tìm kiếm thường lớn hơn lợi ích. Điểm quan trọng hơn: kết quả gắn chặt với phần cứng dùng để tìm. Một kiến trúc NAS tối ưu cho GPU máy chủ có thể không tối ưu chút nào trên NPU của điện thoại.

E. Nguyên tắc bao trùm: đo trên phần cứng đích

Mỗi kỹ thuật trong chương phải kèm phép đo độ trễ thật trên đúng phần cứng sẽ chạy. FLOPs là đại diện tồi, và điều tệ hơn là nó sai *theo hướng khó đoán*: một mô hình ít FLOPs hơn có thể chậm hơn vì nó có nhiều op nhỏ hơn, vì bố cục bộ nhớ của nó không hợp với kernel sẵn có, hoặc vì nó dùng một toán tử mà runtime phải chạy trên CPU.

Sản phẩm cuối của một chu kỳ nén không phải một mô hình mà là một *đường Pareto* như Hình 1.2: tập các điểm mà không điểm nào bị điểm khác trội hơn ở cả hai trục. Chỉ khi có đường này bạn mới chọn được một cách có lý do. Câu hỏi lúc đó không còn là “mô hình nào tốt nhất”, mà là “đổi bao nhiêu độ chính xác lấy bao nhiêu mili-giây”. Câu hỏi thứ hai có câu trả lời.



Hình 1.2. Sản phẩm của một chu kỳ nén (số liệu minh họa). Các dấu chéo là những cấu hình bị trội — vừa chậm hơn vừa kém hơn một cấu hình khác — và chúng bị loại bỏ ngay, không cần thảo luận. Quyết định chỉ còn là chọn một điểm trên đường, và ngân sách mili-giây quyết định điểm nào.

1.4 Thực hành

Repo và tài nguyên. Lượng tử hoá và sparsity: [pytorch/ao](#) (torchao), [NVIDIA/TensorRT-Model-Optimizer](#), [openvinotoolkit/nncf](#), [intel/neural-compressor](#), [bitsandbytes](#). Tỉa: [VainF/Torch-Pruning](#). Chưng cất cho thị giác: công thức có sẵn trong [timm](#); ngoài ra nên đọc các bài chưng cất gần đây trong chính lĩnh vực của bạn, vì chi tiết phụ thuộc bài toán nhiều hơn người ta tưởng. Danh sách này chốt tại tháng 9/2026 và tên các bộ công cụ dẫn đầu sẽ đổi trong 6–12 tháng; các repo hạ tầng (torchao, TensorRT, OpenVINO) bền hơn hẳn tên của thuật toán thời thượng.

Câu hỏi kiểm tra hiểu. Vì sao tỉa không cấu trúc đạt sparsity 90% mà không nhanh hơn trên GPU? Một calibration set 100 ảnh chọn sai thì hỏng theo cơ chế nào, và triệu chứng nhìn thấy được là gì? Khi nào chưng cất thắng lượng tử hoá, và ngược lại?

Mini project (vài giờ đến hai ngày). Lấy một classifier và áp lần lượt bốn bước: giảm độ phân giải 20%, đổi sang backbone hiệu quả, PTQ INT8, rồi QAT. Sau *mỗi* bước đo cả độ chính xác lẫn độ trễ thật trên phần cứng đích. Vẽ đường Pareto và chọn một điểm, kèm một đoạn giải thích vì sao chọn nó. Phần đáng giá nhất của bài này là phát hiện ra bước nào không cho lợi ích như bảng hứa hẹn — và tìm ra lý do.

1.5 Giới hạn và trường hợp hỏng

- **Mọi con số trong chương phụ thuộc phần cứng, và phụ thuộc mạnh.** Một kết quả đo trên GPU máy chủ không dự báo được gì về NPU điện thoại hay về DLA của Jetson. Chuyển chip thì phải đo lại từ đầu, không phải điều chỉnh.
- **Độ chính xác trung bình che giấu thiệt hại tập trung.** Lượng tử hoá và giảm độ phân giải thường lấy đi hiệu năng ở đúng những nhóm hiếm — vật nhỏ, lớp ít mẫu, điều kiện chiếu sáng bất thường. Nếu bạn chỉ theo dõi một con số tổng, bạn sẽ không thấy.
- **Nén nhiều bước không cộng tuyến tính.** Lượng tử hoá một mô hình đã tĩa thường tệ hơn dự đoán, vì tĩa làm phân bố trọng số lệch hơn và do đó khó lượng tử hoá hơn. Thứ tự áp dụng cũng là một siêu tham số.
- **Chưng cất thừa hưởng cả lỗi của teacher.** Student học cả những chỗ teacher sai một cách tự tin, và vì nó nhỏ hơn nên nó không có đủ năng lực để tự sửa.
- **Benchmark ngắn nói dối trên thiết bị bị giới hạn nhiệt.** Đây là chủ đề của chương 21, nhưng nó ảnh hưởng ngược lại mọi phép đo trong chương này.

CẠM BÃY

FLOPs không phải độ trễ, và tin vào FLOPs là lỗi tốn thời gian nhất trong chương. Ba tình huống nó sai theo hướng ngược nhau: một mô hình ít FLOPs với hàng trăm op nhỏ có thể chậm hơn một mô hình nhiều FLOPs với vài op lớn, vì chi phí điều phối trội hơn chi phí tính; một tích chập tách theo chiều sâu có FLOPs rất thấp nhưng tỉ lệ tính trên byte cũng rất thấp, nên nó memory-bound và không nhanh như bảng hứa; và một toán tử mà runtime không hỗ trợ sẽ bị đẩy về CPU, nuốt trọn mọi lợi ích mà chẳng xuất hiện trong bất kỳ phép đếm FLOPs nào.

CẦN NHỚ

Thứ tự đúng là: giảm độ phân giải → đổi kiến trúc → lượng tử hoá → chưng cất → tĩa; và mỗi bước phải kèm một phép đo độ trễ thật trên phần cứng đích, ở chế độ nguồn và nhiệt mà bạn sẽ thực sự chạy. Sản phẩm cuối không phải một mô hình mà là một đường Pareto — chỉ khi có nó bạn mới đang *chọn* thay vì *đoán*. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Độ phân giải	Kích thước ảnh đưa vào mạng; chi phí tỉ lệ với số điểm ảnh. Hạ từ 640 xuống 512 cắt 36% chi phí bằng một dòng config, đổi lại mất vật nhỏ.
••• Memory-bound so với compute-bound	Memory-bound là bị chặn bởi băng thông, compute-bound là bị chặn bởi năng lực tính. Phần lớn mạng thị giác memory-bound, và đó là lý do lượng tử hoá có tác dụng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Lượng tử hoá	Biểu diễn trọng số và kích hoạt bằng số nguyên ít bit, thường INT8. Giảm số byte phải nạp; cái giá là sai số làm tròn tích lũy qua các tầng.
●●● Vì sao calibration set sai làm tụt độ chính xác?	Nó quyết định dải giá trị mà hệ số tỉ lệ phủ. Lấy toàn một điều kiện chiếu sáng thì dải hẹp hơn thực tế, nên mọi giá trị ngoài dải bị cắt cụt khi triển khai.
●● PTQ	Lượng tử hoá sau huấn luyện, chỉ cần calibration set và vài phút. Luôn thử trước; chỉ chuyển sang QAT khi nó tụt quá ngưỡng bạn đặt trước.
●● QAT	Mô phỏng làm tròn ngay trong lúc huấn luyện để mạng học cách chịu đựng. Giữ độ chính xác tốt hơn ở bit thấp; cái giá là một chu kỳ huấn luyện đầy đủ.
●●● Per-tensor so với per-channel	Per-tensor dùng một hệ số tỉ lệ cho cả tầng, per-channel dùng một hệ số cho mỗi kênh. Per-channel gần như luôn đúng cho trọng số conv và linear.
●● Vì sao một kênh ngoại lai đủ phá per-tensor?	Hệ số tỉ lệ đặt theo $\max w $. Với ngoại lai 2,0, một trọng số 0,1 chỉ còn sáu mức — chưa tới ba bit hiệu dụng cho phần lớn tầng.
●●● Vì sao tia không cấu trúc không cho tốc độ thật?	Số không rải rác không giảm số byte phải nạp; bạn còn phải nạp thêm chỉ số để biết chỗ nào là không. Ngoài ra kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa.
●● 2:4 sparsity	Trong mỗi nhóm bốn trọng số, đúng hai bằng không. Cho tốc độ thật, nhưng chỉ trên phần cứng có đơn vị sparse tensor.
●●● Chưng cất	Huấn luyện student nhỏ bắt chước phân phối mềm của teacher thay vì học nhãn cứng. Giữ được nhiều chất lượng nhất trên mỗi MB; student thừa hưởng cả chỗ teacher sai.
●● Vì sao phân phối mềm giàu thông tin hơn nhãn cứng?	Nó mang quan hệ giữa các lớp: sói gần chó hơn mèo. Nhãn cứng vứt đúng phần thông tin đó đi.
●● Tái tham số hoá	Huấn luyện với khối nhiều nhánh rồi gộp chính xác thành một tích chập đơn khi triển khai. Được cả động lực học tốt lẫn đồ thị suy luận gọn.
● NAS	Tự động tìm kiến trúc. Phần lớn dự án không nên dùng: chi phí tìm thường lớn hơn lợi ích, và kết quả gắn chặt với phần cứng dùng để tìm.
●●● Đường Pareto	Tập các cấu hình mà không cấu hình nào trội hơn ở cả độ chính xác lẫn độ trễ. Đây mới là sản phẩm của một chu kỳ nén, không phải một mô hình đơn lẻ.
●● Tia 90% mà độ trễ không đổi?	Bạn đã tia không cấu trúc. Số không rải rác không giảm số byte phải nạp; chỉ tia cả kênh hoặc mẫu 2:4 mới cho tốc độ.
●● Mô hình ít FLOPs hơn nhưng chậm hơn trên Jetson?	Ba nghi phạm: quá nhiều op nhỏ nên chi phí điều phối trội; tích chập tách theo chiều sâu memory-bound; hoặc một op không được runtime hỗ trợ nên rơi về CPU.
●● Lượng tử hoá xong, độ chính xác trung bình không đổi mà hệ thống vẫn tệ đi?	Thiệt hại tập trung ở nhóm hiếm: vật nhỏ, lớp ít mẫu, điều kiện chiếu sáng bất thường. Phải đánh giá theo nhóm mới nhìn thấy.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
TensorRT, pytorch/ao	Lượng tử hoá per-channel, hợp nhất kernel, chọn bố cục bộ nhớ — gần như toàn bộ mạch của chương	Hạ tầng; API đổi nhưng ý tưởng bên dưới ổn định
Sparsity 2:4 trên Tensor Core	Tia bán cấu trúc: ràng buộc mẫu đủ đều để phần cứng khai thác được	Ví dụ rõ nhất cho luận điểm “tốc độ đến từ cấu trúc, không từ tỉ lệ số không”

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Dòng MobileNet, EfficientNet, RepVGG	Kiến trúc hiệu quả và tái tham số hoá — đẩy chi phí sang lúc huấn luyện	RepVGG là minh hoạ sạch nhất; ý tưởng gộp nhánh nay xuất hiện lại trong nhiều detector thời gian thực
Chưng cất mô hình nền tảng xuống student nhỏ	Dùng teacher đóng băng làm nguồn phân phối mềm để đưa mô hình lớp SAM hoặc DINO lên thiết bị	Quan sát thực hành phổ biến 2025–2026; tôi không kiểm chứng được con số cụ thể
bitsandbytes, weight-only INT4	Chỉ nén trọng số khi nút thất là băng thông đọc trọng số chứ không phải kích hoạt	Là chuẩn mực khi phục vụ LLM trên GPU đơn

MINI PROJECT SLAM · INT8 hoá tầng mô tả và đo đủ ba trục · 1 ngày

Mục tiêu	biến “lượng tử hoá thì nhanh hơn” thành ba con số cụ thể trên phần cứng của bạn. Rồi tự quyết định cuộc đánh đổi có đáng ở ngân sách mili-giây của bạn hay không.
Dữ liệu	mô hình mô tả từ mini project chương 19 (hoặc một mô hình mô tả có sẵn nếu bạn bỏ qua bước đó). Calibration set lấy từ một chuỗi <i>khác</i> với chuỗi đánh giá, để không tự lừa mình.
Các bước	đo baseline fp32 trên đủ ba trục: độ chính xác ghép cặp, độ trễ p50 và p99 trên Orin ở đúng chế độ nvmodel sẽ triển khai, và dung lượng tệp trọng số. Rồi lần lượt PTQ per-tensor, PTQ per-channel, và QAT nếu PTQ tụt quá ngưỡng bạn đặt trước. Sau <i>mỗi</i> bước đo lại đủ ba trục, không suy đoán từ bước trước.
Xong khi	một bảng bốn dòng — fp32, PTQ per-tensor, PTQ per-channel, QAT — nhân ba cột: độ chính xác ghép cặp, p99, dung lượng. Kèm một câu kết luận <i>có số</i> : “đổi x điểm ghép cặp lấy y ms và z MB, đáng hoặc không đáng ở ngân sách của tôi”. Nếu per-tensor tụt mạnh hơn per-channel, chỉ ra được kênh ngoại lai nào gây ra điều đó.
Nối với	D/20 cho bảng thứ tự đòn bẩy và ví dụ per-tensor so với per-channel; D/21 cho kỷ luật đo (warm-up, phân vị, tải nhiệt) mà mọi con số ở đây phải tuân theo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì phần cứng không tiết kiệm được gì từ các số không rải rác: bạn vẫn phải nạp dữ liệu và nạp thêm chỉ số để biết chỗ nào là không, kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa, và bản thân chỉ số cũng ăn băng thông. Tốc độ chỉ đến khi cấu trúc đủ đều — 2:4 hoặc tía cả kênh.
- A2. Vì nó đảo ngược được trong một dòng config và cho lợi ích gần như bậc hai theo cạnh ảnh: 640 → 512 cắt 36% chi phí ngay. Nguyên tắc xếp thứ tự là chi phí của việc *sai*, không phải độ tinh vi: sai ở đây tốn nửa giờ, sai ở chưng cất tốn một tuần.
- A3. Vì một hệ số tỉ lệ per-tensor được đặt bởi $\max |w|$. Với ngoại lai 2,0 thì $s = 2,0/127$, nên một trọng số 0,1 chỉ được ánh xạ thành mức 6: toàn bộ dải bình thường còn sáu mức, chưa tới ba bit. Per-channel cho mỗi kênh một s riêng và trả lại đủ 8 bit.
- A4. Ít nhất ba cách. Một: nhiều op nhỏ làm chi phí điều phối trội hơn chi phí tính. Hai: tích chập tách theo chiều sâu có FLOPs thấp nhưng tỉ lệ tính trên byte cũng thấp, nên nó memory-bound. Ba: một toán tử runtime không hỗ trợ bị đẩy về CPU, nuốt trọn lợi ích mà không xuất hiện trong bất kỳ phép đếm FLOPs nào.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn phải nhét một tủ sách vào một chiếc vali có hạn cân. Đó là toàn bộ chương này: mô hình tốt nhất của bạn không vừa cái máy mà nó phải chạy trên. Có năm cách giảm cân, và thứ tự quan trọng hơn kỹ thuật của từng cách. Rẻ nhất là bỏ bớt những cuốn bạn không đọc — tương đương với việc đưa ảnh nhỏ hơn vào máy; làm trong một phút, hồi hận thì làm lại ngay. Kế đó là mua bản bìa mềm thay bìa cứng: cùng nội dung, ít giấy hơn. Rồi mới

đến in chữ nhỏ lại một chút, tức ghi mỗi con số bằng ít chữ số hơn. Cách này rất hiệu quả. Nhưng chỉ cần một trang có cỡ chữ khổng lồ là cả cuốn phải in nhỏ theo, và mọi trang khác thành khó đọc. Sau nữa là nhờ người đọc hộ rồi viết bản tóm tắt. Cuối cùng, cách nghe hay nhất mà thường vô ích nhất: xé bớt chữ trên trang. Bạn xoá được chín phần mười chữ, nhưng vẫn phải mang đủ số tờ giấy, nên vali chẳng nhẹ đi.

Cái giá thì luôn có, và nó không rơi đều: thứ mất đi trước tiên là những cuốn hiếm, những chi tiết nhỏ, những trường hợp bạn ít gặp nhưng lại quan tâm nhất. Và con số duy nhất đáng tin là cái cân ở sân bay, không phải con số bạn tự ước ở nhà.

Một câu: Nén mô hình là xếp vali: làm theo thứ tự rẻ tiền trước, và chỉ tin cái cân ở sân bay chứ đừng tin con số bạn ước ở nhà.

1.6 Kết nối

Chương này là ứng dụng trực tiếp của B/11-gioi-han-tinh-toan: mọi kỹ thuật ở đây chỉ là cách di chuyển trên biểu đồ *roofline*, và biết mình đang ở nhánh nào của biểu đồ đó quyết định kỹ thuật nào có tác dụng. Nó song song với C/18/02-kinh-te-hoc-pretraining: cùng một logic ngân sách, chỉ khác là cột chi đổi từ compute huấn luyện sang compute **suy luận** (*inference*) — và tái tham số hoá cùng chứng cất chính là hai cách chuyển tiền giữa hai cột đó. Nó là tiền đề bắt buộc của D/21-inference-va-trien-khai, nơi mô hình đã nén được đem đi *export* và đo thật, và nơi bạn sẽ phát hiện rằng phần lớn thời gian không nằm ở mô hình. Cuối cùng nó phụ thuộc B/07-chia-du-lieu-va-danh-gia: không có tập đánh giá phân tách theo nhóm hiếm thì bạn không nhìn thấy cái giá thật của việc nén.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao tia không cấu trúc cho sparsity 90% mà không nhanh hơn trên GPU phổ thông, và ba lý do đó lý do nào là lý do phần cứng thật sự?
2. Trong ví dụ per-tensor với một kênh ngoại lai, phần lớn trọng số còn lại bao nhiêu bit hiệu dụng, và per-channel sửa điều đó bằng cách nào?
3. Vì sao giảm độ phân giải đứng đầu danh sách dù nó là kỹ thuật “thô” nhất, và cái giá của nó thường bị bỏ sót ở đâu trong quy trình đánh giá?
4. Khi nào chưng cất thăng lượng tử hoá, và khi nào ngược lại — lập luận qua ràng buộc nào đang chặn bạn?
5. Một mô hình có ít FLOPs hơn nhưng chạy chậm hơn trên Jetson: ba nguyên nhân khả dĩ nhất là gì?
6. Vì sao lượng tử hoá một mô hình đã tia thường tệ hơn dự đoán, và điều đó nói gì về việc coi các kỹ thuật nén là độc lập với nhau?

Nguồn

- [1] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [2] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [3] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [4] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [5] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [6] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [7] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [8] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.